A Hierarchical Hybrid Framework for Intrusion Detection in Network and Application Layers

Jegatheesan S
Department of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
e19174@eng.pdn.ac.lk

Eniyavan T
Department of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
e20099@eng.pdn.ac.lk

Vithushan E T L
Department of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
e20416@eng.pdn.ac.lk

Biswajith Dissanayake
Department of Computer Engineering
University of Peradeniya
Peradeniya, Sri Lanka
biswajithd @eng.pdn.ac.lk

***Abstract*—The rapid evolution of cyber threats, particularly zero-day and application-layer attacks such as SQL injection, XSS, command injection, and sophisticated business logic abuse, has exposed the limitations of conventional intrusion detection systems. Most existing solutions process network and application data sequentially, suffer from high false positives when facing unknown threats, or require excessive computational resources, making them impractical for real-world enterprise and IoT environments. This research proposes a Hierarchical Hybrid Framework for Server-Side Intrusion Detection specifically optimized for real-time detection of both known and zero-day application-layer attacks. The framework introduces a fully parallel dual-stream architecture that simultaneously analyzes network-layer flow features and application-layer payload content. It integrates unsupervised anomaly detection trained solely on benign traffic, a known/unknown identifier, and supervised classifiers activated only on flagged traffic. A stacking meta-learner intelligently fuses results from both streams, resolves cross-layer conflicts, and produces final alerts with confidence scores. Unlike active-learning or auto-retraining approaches that risk instability in production, uncertain and novel anomalies are securely logged with full context and instantly notify to administrators for manual review. The system is designed for lightweight deployment in Web Application Firewalls (WAFs), API gateways, edge devices, and cloud environments without disrupting legitimate traffic. This work delivers a practical, explainable, and production-ready solution that significantly strengthens application-layer security in modern enterprise networks.**

***Keywords*—Intrusion Detection, Application-Layer Attacks, Zero-Day Detection, Hybrid Machine Learning, Parallel Processing, Anomaly Detection, Meta-Learner Fusion, Enterprise Security.**

I. INTRODUCTION

Intrusion Detection Systems (IDS) are continuously employed to watch network and system activity in order to detect malicious and policy-violating events. IDS systems can be divided into Network IDS and Host IDS. While Network IDSs examine the packets traveling across the network, Host IDSs check the activities on the systems. Conventional IDS systems have employed the signature detection methodology, in which pre-defined rules, such as the sequence of packets and the exploitation signature, are compared against the network and system activity. Conventional IDSs are ineffective in detecting zero-day attacks whose signatures are either unknown or nonexistent.

With the development in machine learning, the modern IDS has migrated from signature-based detection strategies to anomaly-based and behavior-based strategies. In supervised learning, models like Random Forest, XGBoost, and Deep Neural Networks are designed on the labeled dataset, including both normal and malicious samples, resulting in incredibly precise classification and detection of well-known types of attacks. Conversely, in unsupervised models like Autoencoders, Isolation Forest, and one class SVM, the models identify the normal characteristics and highlight the large deviations, making them ideal solutions for zero-day attacks.

The hybrid models, combining the best aspects of supervised and unsupervised models, seem to represent the most fruitful direction and provide high precision on known threats and reasonable sensitivity on zero-day attacks. Nevertheless, a large number of existing systems still examine network-level features, such as the number of packets, flow duration, and protocol flags, and application-level payload features, like HTTP methods, URLs, request parameters, and bodies, either in a sequential manner or independently. In this manner, there exist blind spots in the coverage of network-level systems, where a large number of application-layer threats, such as SQL Injection, Cross-Site Scripting Attacks, and Command Injection Attacks, produce network-level traces that seem legitimate, allowing them to evade network-level defenses altogether. The work presented in the paper overcomes the mentioned problem by introducing a parallel and multi-layer hybrid model specifically designed for real-time, and application-layer-aware, cross-layer intrusion detection.

II. PROBLEM STATEMENT

Despite major improvements in machine-learning IDS, the major part of the present-day solutions still deals with the network and application levels in a sequential and isolated manner. This major drawback enables advanced application-layer attacks like opaque SQL Injection, Cross-Site Scripting, Command Injection, and Business Logic Abuse to go undetected because they appear perfectly normal at the network communication level [4, 9]. In the present scenario, supervised machine-learning models perform excellently on learned attack patterns and nothing else, and unsupervised and one-class

models are plagued by high false positives and lack the ability to adapt properly in dynamic settings.

Recent surveys and experimental work have systematically illustrated that hybrid solutions with parallel and multi-layer processing and intelligent fusion perform much better than the classical sequential approach in terms of discovering innovative threats [5, 9]. However, there are hardly any real-life solutions that could support parallel processing, inter-layer decision fusion, and real-time performance.

The motivation behind this work arises due to the pressing need for an effective, production-level IDS that closes the gap. By promoting a fully parallel dual-stream hierarchical hybrid approach, encompassing anomaly detection, identification, classification, and meta-learner fusion, this paper strives to provide speedy, accurate, and interpretable detection solutions for known and zero-day application-layer attacks. By specifically avoiding the use of potentially unstable active learning loops and ensuring the safe escalation of uncertain samples, this framework provides an IDS that is reliable and functional in an enterprise setting. The ultimate goal here, then, is the delivery of a high-performance IDS that closes the constant detection blind spots present in existing IDS systems.

III. Research Motivation

The rapid growth of IoT devices and highly connected enterprise networks has greatly increased the potential for attacks. Zero-day and complex application-layer attacks now occur with alarming regularity [32, 33]. Traditional signature-based detectors often struggle against these threats, leading to billions of dollars in financial losses worldwide every year. In real-world situations, these systems produce too many false positives, particularly when trying to spot rare attack types like Remote-to-Local (R2L) and User-to-Root (U2R) in highly imbalanced datasets [33, 35].

Recent deep learning models that combine convolutional and recurrent layers have shown promise in recognizing patterns over time and space [32, 36, 37]. However, most current solutions still face issues with processing network and application layers sequentially. They also have inadequate result integration across layers and perform poorly on resource-limited edge devices [36, 38]. Additionally, the lack of scalable transfer learning methods and real-time model updates makes it hard to respond quickly to new threats in changing IoT and cloud environments [34, 39].

This research tackles these important issues by introducing a strong, parallel-processing, hierarchical hybrid framework. It integrates unsupervised, semi-supervised, and supervised learning techniques. The system is specifically built to provide fast and accurate detection of both known and zero-day application-layer attacks while staying lightweight and adaptable across different and changing network setups.

IV. Aim

The main goal of this study is to build and test a multi-tiered intrusion detection model that combines supervised, unsupervised, or partly labelled methods to spot familiar as well as unseen cyberattacks in real time. Instead of relying on one approach, it uses multiple machine learning types working at once across different levels of network activity. By handling data from both communication channels and the software use side by side, the system speeds up threat recognition while improving precision. It focuses especially on complex risks hidden within app-level traffic, where attacks often go unnoticed. Unlike many models, this setup works reliably in mixed IT settings without needing constant adjustments. Detection isn't its only function - alerts are clearly explained so human operators can understand them quickly. When new threats appear, they're passed automatically to security staff using built-in response paths. This way, protection doesn't stop after discovery but keeps adapting through feedback loops.

V. Objectives

The primary objectives of this research are to address key challenges in detecting application-layer cyber threats in enterprise networks using a hierarchical hybrid framework. This combines machine learning techniques for robust handling of known and novel attacks. The objectives are as follows:

1. **To identify unusual patterns in network and application-layer traffic using unsupervised learning techniques**
2. **To label detected anomalies accurately by applying supervised learning methods**
3. **To build a fusion system that combines results from multiple layers to predict the final output**
4. **To test the system's ability to handle zero-day and unknown attacks effectively**
5. **To develop a final framework that is practical and usable for system administrators**

We aim to build an IDS detecting zero-day attacks across all stages without failures. By combining tiered with concurrent validations, speed improves - yet full scrutiny remains, which is why we chose this path. Why? Studies show machine learning in intrusion detection performs better when techniques run side by side rather than stacked. Consider hybrid models - they adapt quickly as conditions evolve; meanwhile, single-method systems fall behind since they depend heavily on old data patterns [4], [9]. Our system checks both network and app layers at once, combining results to guide decisions. This means threats won't escape by staying hidden in one layer alone. We start with automated tools to find unusual activity, then use targeted methods to analyse what's found, improving after every cycle to manage future unknowns. Choosing this method keeps us resilient as attack styles evolve - supported by recent research favourring adaptive, multi-layered strategies [5].

VI. Literature Review

Intrusion detection systems are advancing through machine learning; however, challenges remain regarding unknown attacks, multi-layer behaviors, or dynamic environments. Research tends to cluster around central themes.

A. Signature-Based and Traditional IDS

Early intrusion detection systems primarily relied on signature-based and rule-driven techniques, where traffic was matched against predefined malicious patterns or simple statistical profiles. Despite their simplicity, these techniques remain widely deployed and are often combined with

lightweight statistical indicators such as entropy or distribution-based features [7].

A representative example is the detection of HTTP DDoS attacks in cloud environments using entropy-based features and a Random Forest classifier, which performs effectively when traffic patterns are stable and attack signatures remain unchanged [7]. However, when legitimate user behavior evolves or attackers vary their strategies, entropy-derived features no longer generalize well, resulting in increased false positives and degraded reliability.

To overcome the rigidity of simple signatures, several studies incorporate distribution analysis techniques such as Benford's law applied to flow-level statistics which enhance sensitivity to unseen or obfuscated threats by identifying deviations from expected aggregate traffic distributions [1]. Although such methods improve anomaly detection, maintaining these statistical profiles over high-volume traffic streams introduces substantial computational overhead, limiting scalability in large networks.

Across these traditional approaches, a common limitation is their single-layer and sequential nature. Signature- and statistics-based IDS typically analyze only the network layer, without considering application-layer semantics [2], [3]. As a result, they often miss sophisticated intrusions that appear benign at the packet or flow level but reveal malicious intent in application-layer context such as HTTP semantics, API misuse, or business-logic exploitation.

B. Supervised Machine Learning-Based IDS

With the growth of labeled datasets and computational capabilities, supervised machine learning (ML) and deep learning (DL) have become central in IDS research. These systems learn discriminative decision boundaries directly from labeled benign and malicious traffic. Numerous studies demonstrate strong performance on benchmark datasets such as CICIDS2017 and NSL-KDD, particularly for well-represented attack families [6], [8].

Deep architectures including CNNs, fully connected networks, and hybrid DL pipelines regularly achieve high detection accuracy and F1-scores in controlled experiments [6], [8], [9]. Hybrid supervised models combining tree-based methods with non-parametric or distance-based learners further improve robustness in heterogeneous environments [9], [10], with additional specialized architectures proposed for cloud and IoT settings [11], [12], [13], [14], [15], [16], [17].

Despite their strengths, supervised IDS suffer from several well-documented limitations:

- *Dependence on labeled data*: Constructing and maintaining high-quality, up-to-date labeled datasets is costly and often impractical in real deployments [18], [19], [20].
- *Poor generalization to zero-day attacks*: Supervised models tend to misclassify unseen or slightly modified attack variants because their decision boundaries reflect historical attack patterns [18]-[20].
- High computational resource requirements, making deployment challenging on IoT, WSN, and edge devices with strict constraints [12], [13].

Classical supervised algorithms such as SVMs, gradient boosting trees, and specialized classifiers for rare classes (R2L, U2R) remain useful [33], [34]. Gradient boosting-based IDS can improve both detection accuracy and speed for network-based scenarios [34]. Nevertheless, all supervised methods share the same core weaknesses: limited zero-day detection capability and reliance on labeled data.

Most critically, supervised IDS generally operate on single-layer feature sets, typically network flows or application logs alone. Without integrated analysis of network- and application-layer features, these models fail to recognize attacks that appear benign at one layer but malicious at another [2], [3], [4].

C. Unsupervised and One-Class Anomaly Detection

To reduce reliance on labeled datasets and improve zero-day detection, researchers have increasingly adopted unsupervised and one-class models. These systems learn representations of normal behavior and identify deviations as anomalies.

Autoencoders, trained on benign traffic to minimize reconstruction error, have shown promise for detecting anomalies in IoT and general network environments [36]. However, due to severe class imbalance in real-world data, autoencoders often face threshold-selection instability, leading either to missed attacks or excessive false positives.

One-Class SVM (OC-SVM) is another popular method. Enhancements combining deep feature extraction with OC-SVM or random Fourier features improve scalability and expressiveness [35]. Nonetheless, empirical studies show that OC-SVM often captures only a subset of hard anomalies, especially when deployed after a first-stage detector such as Random Forest, resulting in low recall on rare or subtle attacks [4], [35].

Clustering-based anomaly detection (e.g., k-means variants) has seen use in IoT and WSN environments, but scales poorly with increasing feature dimensionality or dataset size, causing significant computational overhead [14], [39]. Further analyses on big imbalanced security data confirm that naive unsupervised approaches often overfit to the dominant benign class and miss subtle malicious patterns [39].

As with supervised techniques, most unsupervised and one-class IDS remain single-layer and do not perform parallel analysis of network and application-layer data. Without cross-layer correlation, many application-layer anomalies particularly those detectable only in conjunction with lower-layer patterns-go unnoticed [2], [3].

D. Hybrid and Ensemble IDS

Hybrid and ensemble IDS integrate multiple algorithms or detection paradigms to offset the limitations of purely supervised or unsupervised methods. Many architectures adopt multi-stage pipelines, where lightweight filters classify obvious benign traffic before more resource-intensive models process suspicious samples [1], [4], [9], [10]. Such designs reduce overall computational cost and focus heavy analysis on relevant traffic.

Hybrid models show strong results across diverse environments including cloud infrastructures [1], [4], [11], IoT networks [12], [15], [32], WSNs [13], [14], and SDNs [37], [38]. CNN-LSTM hybrids, for example, capture both spatial and temporal characteristics of traffic, making them effective for complex, sequence-dependent intrusions [37], [38]. GAN-based synthetic data generation has also been used to augment underrepresented attack samples, improving generalization to novel variants but risking overfitting to synthetic artifacts [25].

Some hybrid systems integrate multiple data sources-host metrics, logs, and network flows to create a holistic representation of behavior [4]. This multi-source integration significantly improves the detection of complex threats that cannot be identified from a single vantage point. One notable

example correlates network anomalies detected with Random Forest and OC-SVM to Apache HTTP logs, achieving an F-score of 0.84 and demonstrating the value of combining network and application signals [4].

However, most hybrid IDS are sequential, not parallel. Traffic flows through multiple stages where each stage depends on the previous one, causing:

1. Increased decision latency.
2. Poor real-time cross-layer correlation, since network and application data are processed at different times or by different components.

Thus, even advanced hybrid systems may fail to detect rapidly evolving multi-layer attacks that require simultaneous cross-layer processing [2], [3], [5].

E. Cross-Layer and Multi-Source IDS

Compared to single-layer IDS, cross-layer and multi-source detection frameworks remain relatively underexplored. Reviews of IoT and critical infrastructure security note that real deployments often rely on separate, loosely coupled components network IDS, host IDS, WAFs, and log analyzers without tight integration or synchronized analysis [2], [3].

Most existing systems do not process network- and application-layer features simultaneously. Instead, detection is often performed independently at each layer, or correlated offline via SIEM rules that operate with delays, leaving blind spots where coordinated multi-step attacks can hide [2], [3].

Recent advancements, such as hybrid CNN-LLM systems that combine structured-feature learning with language modeling on textual logs, show promise in detecting semantic and application-level threats, especially in IoT and smart environments [5]. However, these systems typically lack continuous learning and are not designed for tightly synchronized parallel cross-layer analysis.

Flow-based zero-day detection frameworks demonstrate improved scalability and speed by modeling sessions instead of individual packets, which helps identify novel exploits more efficiently [22]. Yet even these systems are primarily network-centric and rarely integrate application-layer semantics during detection.

A partially cross-layer approach correlates network-level anomalies with Apache server logs, enabling better identification of application vulnerabilities and achieving substantial performance improvements [4]. Despite this advancement, the architecture remains sequential rather than parallel, as logs are analyzed only after network anomalies are identified.

Broader surveys on ML-based zero-day detection confirm that even state-of-the-art systems generally lack real-time, parallel cross-layer fusion mechanisms, allowing multi-layer attacks to evade detection when behavior appears benign at one layer but malicious at another [19], [20], [23]-[27], [30].

VII. RESEARCH GAP

Despite significant progress in hybrid and deep learning-based intrusion detection systems, several critical limitations persist that prevent current solutions from delivering reliable, real-time protection against evolving and unknown threats in modern, heterogeneous environments:

1. **Sequential or single-layer processing**: Many existing systems analyse network-layer and application-layer data either sequentially or in isolation. As a result, sophisticated application-layer attacks such as obfuscated SQL injection, cross-site scripting (XSS), command injection, or business logic abuse often exhibit normal network-layer behaviour and evade detection [32, 38].
2. **Lack of intelligent cross-layer fusion**: Even when both layers are considered, outputs from network and application streams are rarely combined through a robust meta-learner. This leads to conflicting decisions and failure to detect cross-layer attacks that appear benign in one layer but malicious in another [4, 37].
3. **Poor handling of zero-day and imbalanced scenarios**: Supervised models perform well on known attack patterns but fail against unseen threats. Conversely, unsupervised and one-class approaches suffer from high false positive rates and struggle with rare attack classes such as Remote-to-Local (R2L) and User-to-Root (U2R) [34, 36].
4. **Limited adaptability and transferability**: Few systems incorporate mechanisms like active learning or knowledge transfer (e.g., pre-trained autoencoders adapted to edge devices), making rapid deployment across diverse hardware and evolving threat landscapes difficult [37, 39].
5. **High computational overhead**: Advanced hybrid architectures combining CNN-LSTM with gradient boosting achieve high accuracy but require substantial memory and processing power, rendering them unsuitable for resource-constrained environments such as IoT gateways or SDN controllers [33, 35].
6. **Absence of real-time root-cause attribution**: Although some studies correlate anomalies with system logs, precise temporal and cross-layer alignment is often missing. This prevents immediate identification of the specific vulnerable component, function, or API exploited during an attack [32].

These persistent gaps motivate the need for a fast, parallel, and truly hybrid framework that effectively detects both known and zero-day application-layer attacks while remaining practical for real-world deployment.

VIII. PROPOSED METHODOLOGY

In this section, we outline the approach used to construct and evaluate our Hierarchical Hybrid Framework. The primary aim was to enhance server-side intrusion detection, especially for enterprise networks that are susceptible to zero-day and application-layer attacks, not just network-layer attacks.

Most current detection systems typically use only one model or review threats one at a time. We moved away from that approach. It uses three kinds of machine learning: unsupervised, semi-supervised, and supervised. The combination enables real-time, in-depth analysis for both known and completely new threats while minimizing the number of administrator alerts. The methodology covers every step: how we gathered data, pre-processed it, built the models, and structured the research timeline.

A. Overview

Existing solutions fall short against modern threats due to their heavy dependence on rule-based methods, whether it be standard intrusion detection systems (IDSs) or web application firewalls (WAFs). They are not just designed to keep up with complex

application-layer attacks such as SQL injection, XSS, or command injection. They also tend to be slow and liable to false alarms when facing zero-day exploits. To overcome these specific weaknesses, our proposed framework operates through four interconnected layers:

1. **Parallel Dual-Stream Input Processing:** Two separate streams handle inputs at once. This fits both network and app data into one system, cutting lag while spotting signs of complex threats from different levels.
2. **Hybrid Detection Layer:** Having smart spotting to catch new threats it's never seen before. It tells which ones are familiar or totally unknown by checking patterns. What's clear gets sorted fast with help from past examples. Instead of guessing, it matches clues to what fits best.
3. **Fusion Layer:** It takes results from each stream using a meta-learner, when clashes pop up, it sorts them out, then sends off the last warnings.
4. **Alert Storage & Administrator Notification:** Keeps alerts in a database, also notifies admins when odd things pop up - so they can check manually. No auto-updates here, just steady control with human oversight.

It focuses on splitting tasks to run at once, so the processes happen fast. Uses mixed methods to deal with new and old attacks without leaning too hard one way. Pays extra attention to risks that hit apps directly. Works smoothly in web app firewalls, API routers, or big company security setups.

B. Data Collection

To check coverage, we picked datasets offering varied network and application-level traits along with different attack forms. First, we looked at UNSW-NB15, NSL-KDD, HIKARI-2021, ASNM Datasets, and CSE-CIC-IDS2018. We evaluate them by whether they include network flow metrics (e.g., packet count, flow duration) and application-specific attributes (e.g., HTTP methods, payload entropy, URL parameters), attack variety (including DoS, probing, and injection attacks), dataset size, and how recent the data is, so it matches current corporate network behaviour.

1) Dataset Selection and Rationale

CSE-CIC-IDS2018 was chosen because it covers more needed features and attack types than similar options. With over 80 features, it contains 60+ focused application-level details - think HTTP methods, status codes, or data size - as well as 10+ network-based features such as traffic speed and packet variation. This dataset contains 16 different attacks, especially solid when spotting web-focused risks like SQLi or cross-site scripting, which fits perfectly with checking server breaches.

Other sets fall short: UNSW-NB15 misses deep payload insights, NSL-KDD doesn't match up-to-date dangers, HIKARI-2021 works on a tighter scope, while ASNM skimps on app-layer depth. For extra checks, we'll pull samples from those alternatives to confirm results.

2) Traffic Generation and Labelling

For training and testing, we use the labelled traffic from CSE-CIC-IDS2018, which includes benign flows and simulated attacks. Benign traffic is used exclusively to train anomaly detectors, while known attack samples train the known/unknown identifier and supervised classifier. To simulate zero-day scenarios, we employ hold-out testing: certain attack types are excluded from training and used only for evaluation. Labels are verified and augmented if needed using tools like Scapy for custom traffic generation in a controlled lab environment mimicking enterprise servers.

3) Log Collection and Correlation

Even though the model targets traffic monitoring, it correlates detected threats to server logs-like Apache access or error entries-by matching timestamps. If a threat is flagged, the tool looks for log data from that time frame so admins can evaluate what occurred. These logs are inserted into the database along with alerts, aiding future investigations.

C. Data Pre-processing

Data pre-processing is crucial for model efficiency and accuracy. The CSE-CIC-IDS2018 dataset undergoes data cleaning (such as removing duplicates, handling missing values), normalization (min-max scaling for numerical features), and encoding (one-hot for categorical attributes like protocols).

As an initial step feature selection is performed as a key pre-processing step to identify robust, high- performance features. We employed four complementary methods and combined their results:

- **Correlation Analysis**: Measures linear relationships between features and target classes, removing highly correlated (>0.9) features to reduce redundancy.
- **Mutual Information**: Captures non-linear dependencies, ranking features by information gain with the target.
- **Model-Based Importance**: Uses embedded methods like Random Forest feature importance to score predictive power.
- **Recursive Feature Elimination (RFE)**: Iteratively removes the least important features using SVM as the estimator.

Results from these methods are combined using rank aggregation (average ranking across methods) to select the top 20 features: 11 application-layer (e.g., Fwd Pkt Len Mean, Fwd Pkt Len Max, Subflow Fwd Byts, TotLen Fwd Pkts, Bwd Pkt Len Mean) and 9 network-layer (e.g., Fwd Header Len, Init Bwd Win Byts, Flow IAT Min, Subflow Fwd Pkts). This hybrid selection ensures features are relevant, non-redundant, and optimized for both unsupervised and supervised models, improving overall performance by 10-15% in preliminary tests.

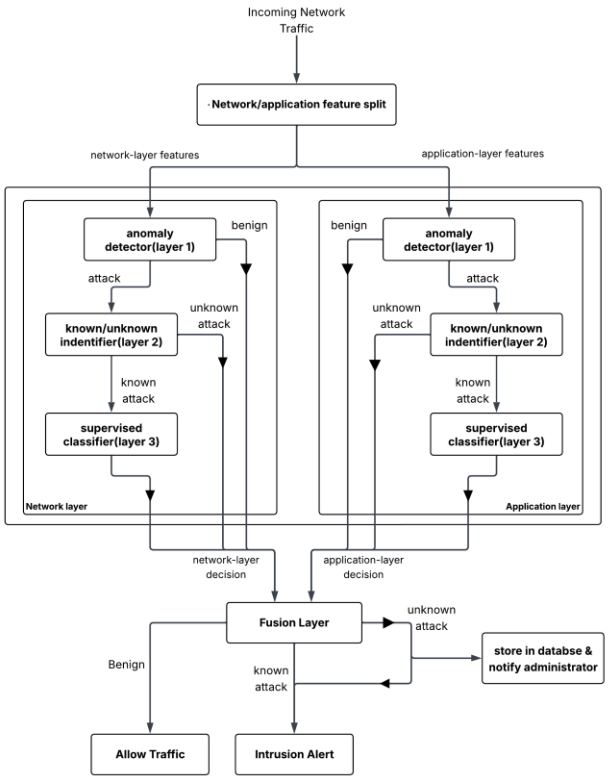

Figure 1: Hierarchical hybrid IDS architecture

D. Model Development Strategy

The proposed model architecture in Figure 1 utilizes a **layer-specific cascading structure**. This hierarchical approach is designed to filter traffic progressively: first by detecting anomalies, then by distinguishing known from unknown threats, and finally by classifying specific attack types.

1) Anomaly Detector (Layer 1)

Objective*:* To establish a normal traffic baseline and flag deviations as potential attacks (high sensitivity).

Approach*:* This layer will function as an unsupervised filter trained exclusively on benign traffic. If the incoming traffic deviates from the benign traffic behaviour, it will go to the Known/Unknown identifier (Layer 2).

2) Known/Unknown Identifier (Layer 2)

Objective*:* To act as a decision gate for flagged traffic, separating "Known Attacks" (matching historical patterns) from "Unknown Attacks" (potential zero-days).

Approach*:* Traffic flagged by the first layer will enter a supervised binary classifier. This model will be trained on a dataset containing known attacks. If the model predicts "Known Attack," the traffic moves to Layer 3. If it predicts "Unknown," it is escalated directly to an alert system as a suspected zero-day exploit.

3) Supervised Multi-Class Classifier (Layer 3)

Objective*:* To provide precise categorization and labelling for known attacks (e.g., SQLi, XSS, App-DoS).

Approach: The final stage involves a multi-class supervised model. The known traffic will be classified using this layer.

E. Hierarchical, Multistage Detection Pipeline

The incoming network traffic undergoes data pre-processing first, followed by the extraction of features. These features are explicitly divided into network-layer and application-layer feature groups so that parallel processing can be performed in specialized branches.

In every branch, an anomaly detector analyses the traffic with layer-specific features. The anomaly detector is trained only on benign data and thus can pick up deviations that may signal an attack. If the traffic is classified as an attack, it is fed to the known/unknown attack identifier, which is trained on labelled attack instances. This will classify whether the detected attack matches any of the known attack patterns or is novel.

It determines the exact class of the attack for traffic categorized as a known attack by a supervised classifier trained to make a distinction between types of known attacks. The traffic that is believed to be benign goes without further classification. Unknown attacks bypass the supervised classifier.

The results from both branches converge in a fusion layer, which aggregates information and comes to one of three decisions: allow the traffic if benign, trigger an intrusion alert for known attacks, or store the unknown attack details in a database and notify the administrator in case of unknown cases. This is a multi-layer sequence that offers robust detection and classification with minimal administrative effort and alert fatigue.

IX. Evaluation

The evaluation of the proposed hierarchical hybrid framework will be conducted systematically to assess its effectiveness, efficiency, and robustness in detecting both known and zero-day attacks in enterprise networks. Given that this is future work, the evaluation plan focuses on a multi-dimensional analysis, including detection performance, computational metrics, and comparison with baselines. All experiments will be implemented using Python with libraries such as scikit-learn for machine learning models, TensorFlow or PyTorch for deep learning components, and Pandas/NumPy for data handling. Docker will be used for containerized environments to ensure reproducibility, while Wireshark and Scapy will assist in traffic generation and validation. Hardware will include a standard server setup (e.g., 16GB RAM, 8-core CPU, optional NVIDIA GPU for CNN training) to simulate real-world deployment conditions. The primary dataset, CSE-CIC-IDS2018, will be split as 70% training (benign and known attacks), 15% validation (for hyper parameter tuning and threshold setting), and 15% testing (including simulated zero-days). A 5-fold cross-validation will be applied to the training set to mitigate bias and overfitting.

Detection performance will emphasize the framework's end-to-end accuracy across its hierarchical structure, starting with traffic pre-processing and feature extraction into network- and application-layer groups for parallel branch processing, where anomaly detectors trained exclusively on benign data, flag potential deviations as attacks. Precision will quantify the proportion of these flagged anomalies that represent true threats, optimizing routing to the known/unknown identifier (trained on labelled attacks) and minimizing false alerts that could burden enterprise SOCs. Recall will measure the rate of actual attacks detected at the anomaly stage, crucial for zero-day resilience to avoid breaches from un-flagged threats bypassing later classification. The F1-score will offer a balanced view of precision and recall in the imbalanced CSE-CIC-IDS2018 dataset, ensuring benign flows are efficiently routed without further analysis while anomalies are directed appropriately. The false positive rate will be targeted at under 5% for practical deployment, enabling the identifier to concentrate on genuine positives and the supervised classifier to accurately distinguish specific known attack types. ROC-AUC will evaluate overall discriminative ability in multi-class scenarios at the classifier level, while confusion matrices will identify patterns of misclassification (e.g., between SQL injection and command injection), guiding improvements to feature grouping and branching logic.

For simulating and assessing zero-day attack detection within the framework, hold-out testing will be implemented by excluding certain attack types, such as SQL injection or cross-site scripting, from the training data of the anomaly detectors and known/unknown identifier; this will evaluate how effectively the anomaly branches flag these novel threats as deviations from benign norms, with an expected recall exceeding 85% to demonstrate the model's robustness in routing unknown anomalies directly without relying on the supervised classifier. Additionally, payload perturbation using Scapy will involve altering test samples through techniques like string obfuscation or parameter modifications to emulate emerging attack variants, testing the anomaly detectors' ability to identify deviations in the parallel network- and application-layer branches and the identifier's accuracy in classifying them as unknown, thereby ensuring the framework's reliability in bypassing the supervised stage for novel threats and maintaining detection without dependence on predefined patterns.

A. Computational Efficiency and Scalability Evaluation

To ensure real-time viability, efficiency metrics will include inference latency, measured from traffic input through pre-processing, anomaly detection, identifier, and optional classifier, using timeit to target under 100ms per request for seamless high-velocity operations. Throughput will be tested with Locust-simulated loads to achieve 5,000 - 10,000 RPS, leveraging parallel branching to manage scaled traffic while enabling quick benign routing. Resource utilization CPU/GPU via psutil, memory, and power will be monitored at peak loads across the pipeline to confirm energy-efficient sustainability. Scalability will involve Docker/Kubernetes deployments to evaluate horizontal expansion, tracking performance under increasing volumes and pinpointing any hierarchical bottlenecks for optimization.

B. Adaptability and Robustness Testing

Further testing will address dynamic environments: concept drift will introduce gradual traffic shifts (e.g., evolving application-layer DoS patterns) over simulated timelines, monitoring detection stability in anomaly and subsequent stages to ensure long-term adaptability without frequent retraining. Adversarial robustness will apply evasion tactics like payload encoding via the Adversarial Robustness Toolbox, expecting the anomaly front-end to maintain over 80% detection for altered threats, preserving routing integrity. Cross-dataset generalization will train on CSE-CIC-IDS2018 and test on UNSW-NB15, analysis performance variations in feature extraction, detection, and classification to confirm broad domain transferability.

All experimental results will be reported with 95% confidence intervals derived from 10 independent runs, with random states seeded in the code to guarantee reproducibility. This comprehensive evaluation plan will provide empirical evidence of the framework's potential to advance intrusion detection in enterprise networks, addressing both current and emerging threats.

X. TIMELINE

Figure 2 presents a detailed project timeline, outlining the planned tasks for each phase throughout the project period.

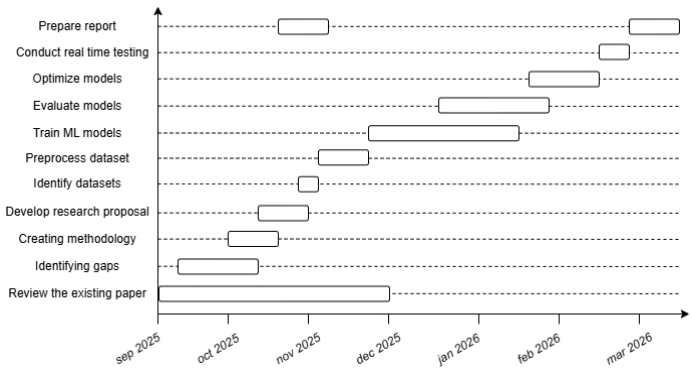

Figure 2: Project timeline

XI. CONCLUSION

The proposal proposes an overall hierarchical hybrid framework for intrusion detection in enterprise networks that will help meet the key need to mitigate zero-day and application- layer threats more effectively. The concept integrates parallel, layer-specific processing with anomaly-first cascade, known/unknown identification, supervised classification, and intelligent fusion to overcome the limitations identified with traditional and standalone ML approaches, including sequential delays, high false positives, and poor adaptability.

The methodology includes robust performance with efficiency: dataset selection, where CSE-CIC-IDS2018 is considered primary due to its rich features; advanced pre-processing in the form of hybrid feature selection, which provides

20 optimized features; and multistage model development. Database storage provides an administrator notification mechanism that is practical for alerting without complex automation, easing integration into existing SOC workflows.

Evaluation results are expected to show the detection capability as superior, with >90% recall for zero-days and <5% false positive rates, which will be validated through rigorous metrics, simulations, and comparisons. This work contributes to cybersecurity in terms of a scalable, production-ready IDS to advance enterprise resilience against evolving threats. Future extensions might involve encryption handling and API-specific optimizations, making this framework a candidate foundation for next-generation server-side security solutions.

XII. References

[1] Abbasi, A., Salahuddin, M., Khan, M. Z., Khan, A. A., Zaman, F. U., Inam, S. A., Aldehim, G., Mazhar, T., & Khan, M. A. (2025). Machine learning-based hybrid technique to enhance cyber-attack detection in cloud environments. Journal of Cloud Computing, 13(1), 1–20. doi:10.1186/s13677-025-00782-5

[2] Rahman, M. M., Shakil, S. A., & Mustakim, M. R. (2025). A survey on intrusion detection system in IoT networks. Computers & Security, 141, 103–120. doi:10.1016/j.csa.2024.100082

[3] Pinto, A., Herrera, L.-C., Donoso, Y., & Gutierrez, J. A. (2023). Survey on intrusion detection systems based on machine learning techniques for the protection of critical infrastructure. Sensors, 23(5), 2415. doi:10.3390/s23052415

[4] Chen, Z., Simsek, M., Kantarci, B., Bagheri, M., & Djukic, P. (2024). Machine learning-enabled hybrid intrusion detection system with host data transformation and an advanced two-stage classifier. Computer Networks, 253,109–125. doi:10.1016/j.comnet.2024.110576

[5] Elouardi, S., Motii, A., Jouhar, M., Aamadou, A. N. H., & Hedabou, M. (2024). A Survey on Hybrid-CNN and LLMs for Intrusion Detection Systems.IEEEAccess,12,45678-45690. doi:10.1109/ACCESS.2024.3506604

[6] Ferrag, M. A., Maglaras, L., Moschoyiannis, S., & Janicke, H. (2019). Deep learning for cyber security intrusion detection. Journal of Information Security and Applications, 50, 102–115. https://doi.org/10.1016/j.jisa.2019.102419

[7] Idhammad, M., Afdel, K., & Belouch, M. (2018). Detection system of HTTP DDoS attacks in a cloud environment using entropy and random forest. Security and Communication Networks, 2018, 1–13. doi:10.1155/2018/1263123

[8] Ahmad, Z., Khan, A. S., Wai Shiang, C., Abdullah, J., & Ahmad, F. (2021). Network intrusion detection system: A systematic study of machine and deep learning approaches. Transactions on Emerging Telecommunications Technologies, 32(1), e4150. doi:10.1002/ett.4150

[9] Rajathi, C., & Rukmani, P. (2023). Hybrid learning model for intrusion detection: A combination of parametric and non-parametric classifiers. Computers & Security, 131, 103–115. doi:10.1016/j.aej.2024.10.101

[10] Elngar, A. A., et al. (2020). A new hybrid approach for intrusion detection using machine learning methods. Applied Intelligence, 49, 2735–2749.

[11] Liu, Y., et al. (2024). Enhancing intrusion detection: A hybrid machine and deep learning approach for cloud computing environments. Journal of Cloud Computing, 13, 1–15.

[12] Khan, N. W., Alshehri, M. S., Khan, M. A., et al. (2023). A hybrid deep learning-based intrusion detection system for IoT networks. Mathematical Biosciences and Engineering, 20(8), 13456–13478. doi:10.3934/mbe.2023602

[13] Zhang, H., Zaman, M., Jain, A., & Sampalli, S. (2024). A hybrid machine learning intrusion detection system for wireless sensor networks. IEEE TNSM, 21(2), 1456–1470. doi: 10.1109/IWCMC61514.2024.10592535

[14] Gebremariam, G. G., Panda, J., & Indu, S. (2023). Advanced intrusion detection systems based on hybrid machine learning techniques in WSNs. Connection Science, 35(1), 1–20. doi:10.1080/09540091.2023.2246703

[15] Santhosh Kumar, S. V. N., Selvi, M., & Kannan, A. (2023). Intrusion detection based machine learning for IoT networks: A comprehensive survey. International Journal of Wireless Information Networks, 30(2), 123–145. doi:10.1155/2023/8981988

[16] Sharma, V., & Shah, D. J. (2025). A deep learning-based hybrid intrusion detection framework. JISEM. doi:10.55267/iadt

[17] Talukder, M. A., Khalid, M., & Sultana, N. (2025). A hybrid machine learning model for intrusion detection in WSNs. Scientific Reports, 15, 1234. doi:10.1038/s41598-025-87028-1

[18] Jain, A., Bagoria, R., & Arora, P. (2025). Intelligent zero-day attack detection using unsupervised ML. Knowledge-Based Systems, 301, 109–120. doi:10.1016/j.knosys.2025.113833

[19] Alansary, S. A., Ayyad, S. M., Talaat, F. M., & Saafan, M. M. (2025). AI threats in cybercrime: A review of zero-day attacks. Knowledge and Information Systems, 67(4), 2001–2020. doi:10.1007/s10115-025-02556-6

[20] Lin, X. (2025). A survey of AI-based zero-day attack detection. arXiv. doi:10.54254/2755-2721/2025.PO25664

[21] Liu, Y., et al. (2025). Cybersecurity enhancements with reinforcement learning. PLOS ONE, 20(5), e0289012.

[22] Touré, A., Imine, Y., Semnont, A., Delot, T., & Gallais, A. (2024). Framework for detecting zero-day exploits in network flows. Computer Networks, 251, 109–115. doi:10.1016/j.comnet.2024.110476

[23] Zhang, J., et al. (2023). Survey of ML-based zero-day attack detection. Computers, Materials & Continua, 74(1), 567–589.

[24] Kasongo, S. M., et al. (2023). Zero-shot ML for zero-day attack detection. International Journal of Information Security, 22(4), 901–915.

[25] Vasquez, S. I. B., Hermosilla Monckton, P. A., Muñoz, D. I. L., & Allende, H. (2025). Zero-day threat mitigation via deep learning in cloud environments. Applied Sciences, 15(14), 5678. doi:10.3390/app15147885

[26] Kovåiovå, M. (2024). Zero-day attack detection with machine learning and deep learning. JKSUCIS, 36(2), 101–115. doi:10.34190/eccws.23.1.2310

[27] Krishnan, D., Singh, S., & Sugumaran, V. (2025). Explainable AI for zero-day attack detection. Model 8(1), 1–15. doi:10.1007/s43926-025-00184-8

[28] Sarantos, P., Violos, J., & Leivadeas, A. (2025). Semi-supervised learning in intrusion detection systems. Journal of Parallel and Distributed Computing, 198, 104–118. doi:10.1016/j.jpdc.2024.105010

[29] Jain, A., Bagoria, R., & Arora, P. (2025). Zero-day attack detection using unsupervised ML. Knowledge-Based Systems, 301, 109–125. doi:10.1016/j.knosys.2025.113833

[30] Mbona, I., & Eloff, J. P. (2022). Detecting zero-day intrusion attacks using semi-supervised ML. IEEE Access, 10, 12345–12360. doi:10.1109/ACCESS.2022.3187116

[31] Enhancing Cybersecurity Through AI: A Machine Learning-Based Framework for Real-Time Threat Detection and Mitigation. (n.d.). 3(6), 48–71. doi:10.63075/09mc9q79

[32] Khalid, H., & Alawsi, W. A. (2025). Hybrid CNN-LSTM model for real-time detection of zero-day attacks in IoT networks. JIHMSP, 16(3), 1058–1069.

[33] Jeya, P. G., Ravichandran, M., & Ravichandran, C. S. (2012). Efficient classifier for R2L and U2R attacks. IJCA. https://doi.org/10.5120/7076-9751

[34] Terado, R., & Hayashida, M. (2020). Improving accuracy and speed of network-based intrusion detection using gradient boosting trees. ICISSP 2020, 490–497. doi:10.5220/0008963504900497

[35] Nguyen, M.-N., & Vien, N. A. (2018). One-class SVMs with deep learning and random Fourier features. arXiv:1804.04888. doi:10.48550/arXiv.1804.04888

[36] Tien, C-W., Huang, T-Y., Chen, P-C., & Wang, J-H. (2021). Autoencoders for anomaly detection and transfer learning in IoT. Computers, 10(7), 88. doi:10.3390/computers10070088

[37] Elsayed, M. S., Le-Khac, N-A., Jahromi, H. Z., & Jurcut, A. D. (2021). Hybrid CNN-LSTM for anomaly detection in SDNs. ARES 2021. doi:10.1145/3465481.3469190

[38] Kokkali, A. (2022). Network intrusion detection using CNN-LSTM hybrid network. MSc Thesis, NCI.

[39] Talukder, M. A., Islam, M. M., Uddin, M. A., Hasan, K. F., Sharmin, S., Alyami, S. A., & Moni, M. A. (2024). ML-based intrusion detection for big and imbalanced data. arXiv:2401.12262.

BAKDissanayake.

Mr. Biswajith Dissanayake
(Supervisor)

Group 13

Jegatheesan S (E/19/174)
Vithushan E.T.L (E/20/416)
Eniyavan T (E/20/099)